

Automatic Creation of Indices in PostgreSQL

Project Submission Report

Name	Roll number
Siddhant Mulkikar	23B0956
Yuvraj Gupta	23B0999
Sandeep Reddy Nallamilli	23B1006
Yash Sabale	23B1043

Public git repository (forked from postgres/postgres):

<https://github.com/HypedUpb0ii/postgres>

The repository is a fork of the upstream PostgreSQL source tree. The initial pull from upstream is preserved as a single ancestor commit; all our work is on top of it under `contrib/auto_index/`, `scripts/`, and `report/`. Per-file diffs against upstream are included in the submission zip under `diffs/`.

1 Motivation

Indexes are the single biggest lever a relational database has on read latency — turning an $O(N)$ filter into an $O(\log N)$ lookup — but they are notoriously hard to manage manually. Workloads change over time, indexes have ongoing write-amplification and storage cost, composite-key ordering is non-obvious ((a, b) or (b, a) ?), and most teams add indexes reactively and never remove them. Oracle 19c ships an “Auto Indexing” feature; SQL Server has Auto Tuning. PostgreSQL has `hypopg` (hypothetical indexes) and `pg_qualstats` (predicate counters), but *no autonomous create/drop loop* — a DBA still has to interpret recommendations and act. Our project closes that gap: a PostgreSQL extension, `auto_index`, that observes the live workload and creates and drops indexes (single-column or composite, up to three columns) entirely on its own.

2 Functionality implemented

The extension hooks into PostgreSQL in three places. An `ExecutorEnd` hook runs after every query, walks the plan tree, finds every `SeqScanState`, and decodes its filter quals into `(attno, op)` pairs. The decoder handles `Var op Const`, `IN (...)`, `BETWEEN`, `AND/OR` via `BoolExpr` (so even TPC-H Q19’s deeply nested OR-of-AND extracts correctly), and implicit casts via `RelabelType`. A fixed-size shared-memory area (`AutoIndexSharedState`) holds per-table predicate counters: per column “singleton” counts and a separate “colset” bucket that records when 2–3 columns co-occur in the same query. A background worker wakes every `check_interval` seconds, folds interval counters into cumulative state, runs the active create-strategy and the drop logic, and emits `CREATE INDEX / DROP INDEX` directly via SPI.

Composite indexes (up to 3 columns). When the same group of 2–3 columns repeatedly co-occurs, the worker builds a composite. Column order at create time puts equality columns first, range columns last (B-tree cannot use later columns past a range), with `attno` as the tiebreak. To avoid redundant work, the singleton index on the composite’s *leading* column is suppressed (subsumed by the composite’s leftmost prefix), but singletons on non-leading columns are kept because B-tree cannot use them on their own.

Pluggable create-strategies. Six strategies are selectable through the `auto_index.create_strategy` GUC:

Strategy	Rule
<code>ski_rental</code> (<i>default</i>)	$\text{cumulative_benefit} \geq \lceil \text{build_cost} / \text{seq_scan_cost} \rceil$, AND read/write ratio above threshold
<code>simple_threshold</code>	$\text{cumulative_benefit} \geq N$
<code>ratio_only</code>	read/write ratio above threshold
<code>cost_gain</code>	projected savings minus maintenance must pay off <code>build_cost</code>
<code>size_gated</code>	<code>simple_threshold</code> + minimum table size
<code>always</code>	fire on any positive benefit

Each strategy is a single `decide_create_*(ctx) → bool` function; the dispatcher is one switch.

Drop logic. Drop is ski-rental in reverse. For every auto-created index (we keep our own catalog table that survives restart) the worker reads `pg_stat_user_indexes.idx_scan` each interval. If `idx_scan` increased, the idle accumulator is reset; otherwise $(\text{writes} \times \text{maint_per_write} + \text{seq_scan_cost}) \times 1000$ is added. Once the accumulator passes `build\_cost` $\times 1000$ the index is dropped. The `seq\_scan\_cost` floor ensures even zero-write tables eventually age out an index that is never read.

SQL surface. `auto_index_reset()`, `auto_index_stats()`, and the catalog view `auto_index_catalog` (with the helper `auto_index_attnames(relid, attnos)` for human-readable column names).

Scope limitations (documented in code, not security-critical): predicate extraction only fires on `SeqScanState` — once any other index exists on a table, the planner switches to `Bitmap Heap Scan` and our hook can no longer see filter predicates (the dominant case in OLTP, capping TPC-C gains around $1.14\times$). Aggregate-bound queries (TPC-H Q01 doing `SUM` over the whole table) cannot be helped by any index in principle.

3 Code created

All new code lives under `contrib/auto_index/` (the extension) and `scripts/` (build and benchmark drivers). *No upstream PostgreSQL source files were modified* — the extension lives entirely in `contrib/`, hooks into existing extension points, and links cleanly against an unmodified server.

Path	Purpose	Li
<code>contrib/auto_index/auto_index.c</code>	core: hooks, shmem, bgworker, parsing, strategies, DDL emission	18
<code>contrib/auto_index/auto_index.h</code>	shared structs and GUCs declaration	68
<code>contrib/auto_index/auto_index-1.0.sql</code>	SQL functions and catalog table	37
<code>contrib/auto_index/auto_index.control, Makefile</code>	extension manifest, PGXS build	—
<code>scripts/{build,run,benchmark,demo}.sh</code>	top-level driver scripts	~2
<code>scripts/lib/*.sh</code>	per-benchmark helpers (pgbench, TPC-C, TPC-H, strategies, setup)	~7
<code>scripts/tpch_queries/q*.sql</code>	8 representative TPC-H queries	—
<code>scripts/tpcc_xn/*.sql, schema/load</code>	TPC-C-lite schema and the four transactions	—
<code>Dockerfile, docker-compose.yaml</code>	reproducible build container	—

The C source is split into six clearly-labelled sections inside `auto_index.c`: 1. `MODULE INIT & GUCs`, 2. `SHARED MEMORY`, 3. `PARSING`, 4. `STRATEGIES`, 5. `INDEX MANAGEMENT`, and 6. `SQL-CALLABLE FUNCTIONS`.

4 How we tested

Four benchmarks, each scripted under `scripts/benchmark.sh`. All numbers come from a Docker container (Ubuntu 22.04, 6 cores, 8 GB RAM, NVMe SSD), PostgreSQL 19devel built with `-O2 -enable-debug -enable-cassert`. Each scenario was run three times.

4.1 pgbench live demo. 300 K-row table, 4 clients $\times$ 2 worker threads, single equality predicate. TPS reported every 2 s. Result: the bgworker fires at the second wake ($t = 6$ s) and TPS jumps from ~ 340 to $\sim 17\,800$ in a single window — a $\sim 52\times$ step, reproducible across all three runs. This is the strongest single demonstration that the extension is alive and correct.

4.2 TPC-C-lite (OLTP). 2 warehouses, pgbench-driven mix of NewOrder / Payment / OrderStatus / StockLevel. The single auto-created index was on `orders.o_c_id` (the OrderStatus join column); `auto_index` correctly *avoided* indexing customer columns despite heavy reads, because Payment also writes to those rows. Mean speedup $1.14\times \pm 0.02\times$ across runs — a small but correct OLTP result, capped by the bitmap-heap-scan limitation listed above.

4.3 TPC-H SF=1 (analytical). 6 M-row `lineitem`, 8 representative queries (Q1, Q3, Q5, Q6, Q10, Q12, Q14, Q19). All six strategies were measured against the same baseline. The most aggressive strategy (`ratio_only`) reached a geometric-mean speedup of $1.79\times$, with $3\times+$ speedups on Q06, Q12, Q19 — driven by the 3-column composite (`l_quantity`, `l_discount`, `l_shipdate`) that exactly matches Q06’s filter. The conservative default (`ski_rental`) reached $1.53\times$ with only 6 indexes built.

4.4 Lifecycle demo. 50 K-row table; phase A is 200 reads on a single column then 200 reads on a 3-column predicate; phase B is 300 INSERT/UPDATE/DELETE rounds with *no* reads on the indexed columns. In every run, phase A produced one singleton plus one composite, and phase B dropped both within ~ 20 s (four bgworker cycles). 3/3 runs PASSED — strongest correctness signal that the extension owns the full create-then-drop lifecycle.

A late-stage docker-compose fix is also worth noting: PostgreSQL parallel hash joins back their shared hash tables with POSIX shm under `/dev/shm`, and Docker’s 64 MB default caused TPC-H Q10/Q12/Q14 to fail with “could not resize shared memory segment”. We bumped `shm_size: 1g` in `docker-compose.yaml` and added a named volume for `/usr/local/pgsql` so future compose-file edits don’t nuke the install.

5 How we used AI

We used **Anthropic Claude** (via Claude Code, models Opus 4.6 / 4.7) extensively as a pair-programmer and a research assistant on PostgreSQL internals. AI was *not* used to design the high-level approach (the ski-rental analogy, the strategy-pluggable architecture, the choice of which hook points to use) — those came from team discussion and the original project proposal — but the line-by-line C, the qual-extraction code, the benchmark scripts, and most of the PostgreSQL-internals research were AI-assisted.

Representative prompts we used. Condensed from real sessions; we kept transcripts in our local notes.

- *Exploration of internals.* “Show me where in the PostgreSQL executor the per-scan filter quals are stored at runtime — I want to read them in an `ExecutorEnd` hook. Point me at the struct field and the file.”
- *Hooking question.* “What’s the right way for a contrib extension to register an `ExecutorEnd_hook` while playing nicely with chained extensions? Show the standard chain-and-restore idiom.”
- *Shared memory.* “I need a fixed-size shared-memory area sized at startup, with LWLocks, that holds an array of 256 per-table records each containing a 32-slot column counter array. Show me the `_PG_init`-time `shm_request_hook + shm_startup_hook` pattern for PG 17/18.”
- *Qual extraction.* “Given a `List *quals` from a `SeqScanState`, write a recursive walker that handles `OpExpr`, `BoolExpr`, `ScalarArrayOpExpr`, and peels `RelabelType`. For each leaf, emit `(varattno, op_kind)` where `op_kind` $\in$ {equality, range}.”
- *DDL safety.* “What’s the right way for a background worker to emit a `CREATE INDEX` (non-concurrent is fine) without breaking the transaction the user is in? SPI? An autonomous connection?”
- *Composite-key ordering.* “B-tree composite key: explain exactly which prefixes a multi-col B-tree can satisfy when the user queries `WHERE a = ? AND b > ? AND c = ?`.”
- *Catalog tracking.* “I want a small extension-owned table that survives `pg_ctl restart` and lists every index this extension has created. What’s the cleanest pattern — install via the SQL install file?”
- *Benchmark plumbing.* “Write a bash script that runs the same TPC-H query set under each of six `auto_index.create_strategy` values, restarts the server between strategies, and emits a comparison table with per-query speedup and geomean.”
- *Debugging.* “PostgreSQL is failing TPC-H Q10 inside Docker with ‘could not resize shared memory segment to 16777216 bytes: No space left on device’. What’s the actual cause and the docker-compose fix?”
- *Report drafting.* “Write the implementation section of a 3-page project report from the following bullet list of facts about the extension. Keep it terse — no headers per paragraph, no bullet-list filler.”

How we worked with AI in practice. (1) *Read first, generate second.* For PostgreSQL-internals questions we asked AI to find and explain the relevant upstream code (e.g. `src/backend/executor/execMain.c` around `ExecutorEnd`) before asking it to generate anything. We then read the upstream file ourselves. (2) *Code was always reviewed and tested.* Every chunk of AI-generated C went through compile + run + regression of an earlier benchmark before being committed. Several AI-suggested approaches were rejected for being non-idiomatic (e.g. an early draft used `palloc` in shared memory; we caught it and rewrote with `ShmemAlloc`). (3) *Benchmarks were authored manually first, then refined with AI.* The timing methodology and the ski-rental drop logic are ours; AI helped flesh out the bash plumbing and the median/geomean post-processing. (4) *The report was AI-assisted.* Both the technical report (`report.tex`) and this submission report were drafted with AI help from team-supplied bullet points and benchmark CSVs, then edited by hand.

We did not use AI for: the project topic / proposal, the choice of strategies, the decision to build a fork rather than a patch, or the specific test-data / benchmark choices.

6 Submission zip layout

```
auto_index_submission/
|-- README.md                # build & run instructions
|-- git_url.txt              # link to public fork
|-- diffs/                   # per-file git diffs vs. upstream PostgreSQL
|   |-- .gitignore.diff      # the only modified upstream file
|-- new_files/               # all new files we authored, mirrored at original paths
|   |-- contrib/auto_index/...
|   |-- scripts/...
|   |-- Dockerfile, docker-compose.yaml
|-- test_data/               # test artefacts
|   |-- tpch_queries/q*.sql, tpch_schema.sql
|   |-- tpcc_schema.sql, tpcc_load.sql, tpcc_xn/*.sql
|-- report/
|   |-- submission_report.pdf # this document
|   |-- report.pdf            # full technical report (deeper dive)
```

The TPC-H raw data files (*.tbl, ~70 MB at SF=0.1) are not shipped — `scripts/lib/tpch_setup.sh` regenerates them with `tpch-dbgen` on first run.

Reproducing the headline results from a clean checkout.

```
git clone https://github.com/HypedUpb0ii/postgres.git
cd postgres
scripts/build.sh all          # docker + pg + init + extension
scripts/benchmark.sh pgbench  # ~52x live step
scripts/benchmark.sh tpch     # 8-query 3-phase TPC-H
scripts/benchmark.sh tpcc     # OLTP
scripts/demo.sh               # full create-then-drop lifecycle, ~90 s
```